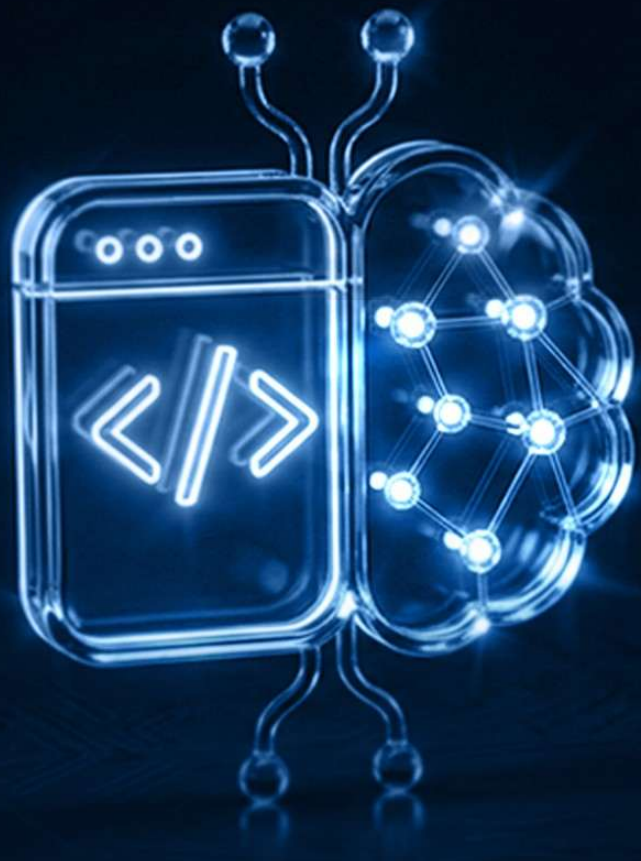




;

GenAI

مهندسی نرم افزار در عصر هوش مصنوعی

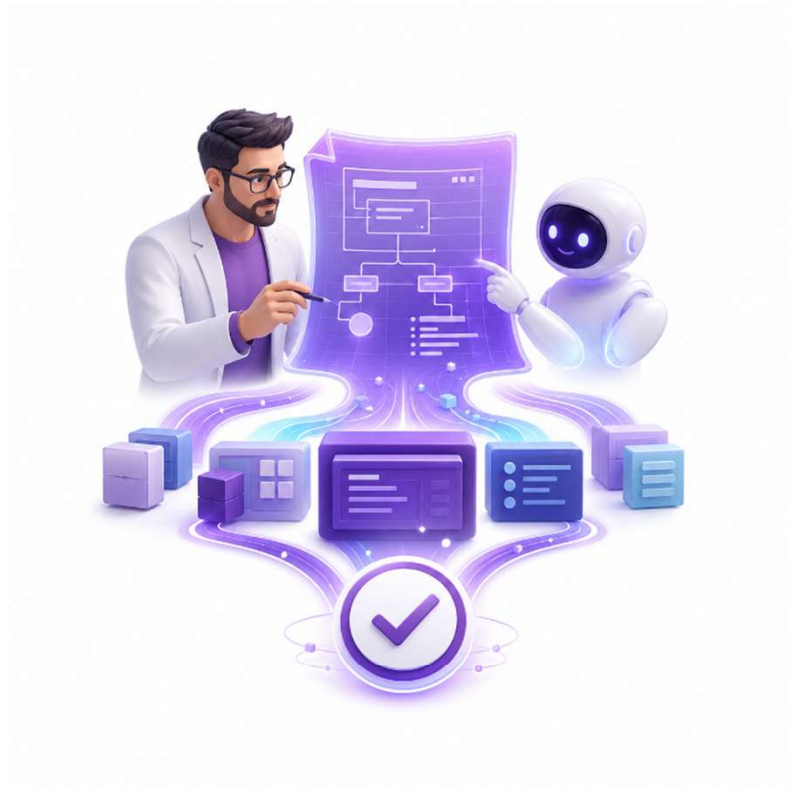
علیرضا ارومند

جلسه دوم

آشنایی با SDD



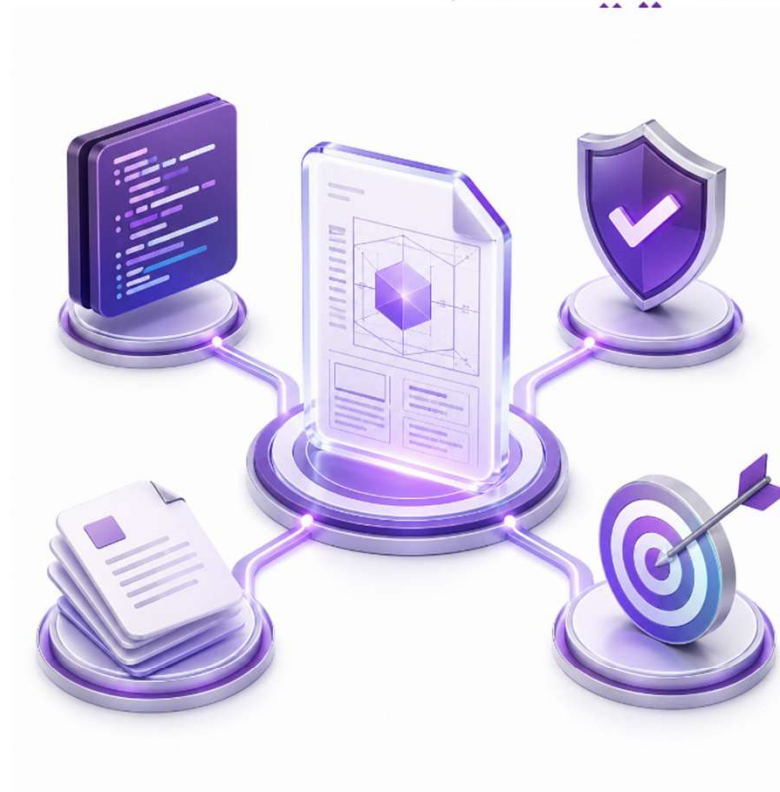
توسعه مبتنی بر مشخصه



- مرور آموزشی مفهوم Spec-Driven Development
- از تعریف مشخصه تا اجرای ساخت‌یافته
- نقش انسان و عامل هوشمند در فرایند توسعه
- تبدیل نیاز محصول به واحدهای قابل اجرا
- اعتبارسنجی خروجی بر اساس رفتار مورد انتظار



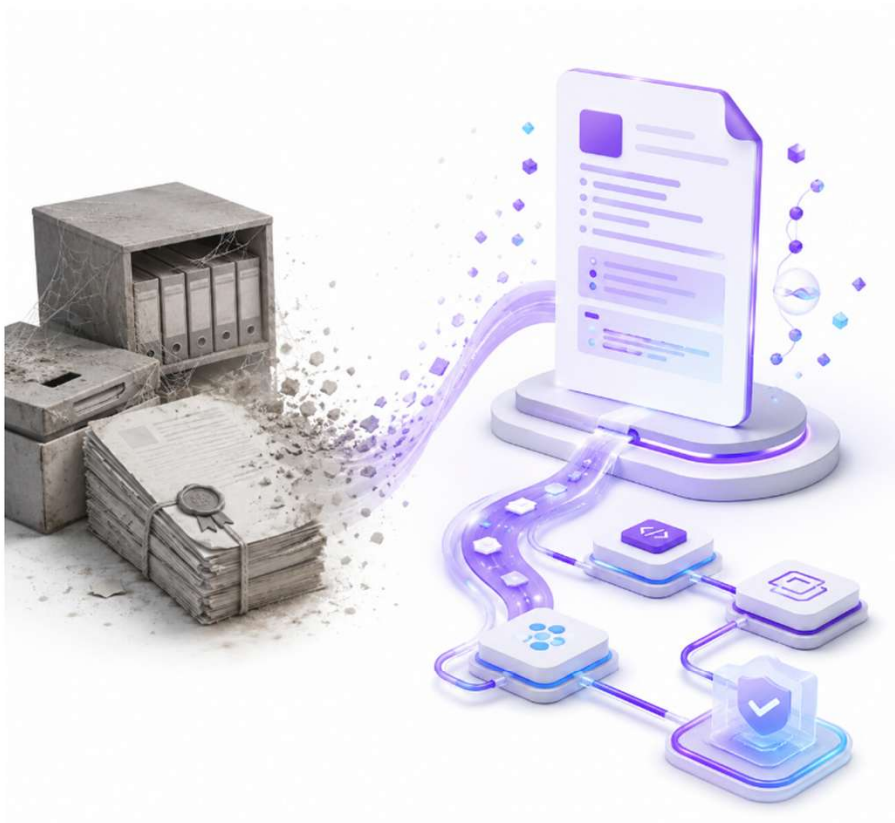
SDD چیست؟



- در SDD، مشخصه مرجع اصلی همه کارهای بعدی است.
- پیاده‌سازی، آزمون، مستندات و اعتبارسنجی از روی آن شکل می‌گیرند.
- برخلاف روال سنتی، کد منبع حقیقت نهایی محسوب نمی‌شود.
- کد خروجی است و مشخصه قرارداد رفتاری محصول است.
- در پایان، محصول با رفتارهای نوشته‌شده سنجیده می‌شود.



SDD چه نیست؟



- SDD مستندات قدیمی و جدا از کد نیست.
- مشخصه کنار کد زندگی می‌کند و همراه آن نسخه‌بندی می‌شود.
- SDD آبخاری نیست؛ مقصد را تعریف می‌کند، نه مسیر ثابت را.
- تغییر مجاز است، اما ابتدا باید در مشخصه ثبت شود.
- SDD سرعت Vibe Coding را حفظ می‌کند و به آن جهت می‌دهد.



چگونه به اینجا رسیدیم؟

- TDD آزمون را پیش از کد قرار داد.
- آزمون‌ها معمولاً سازوکار را بهتر از تجربه کاربر توصیف می‌کنند.
- BDD رفتار را از دید کاربر وارد فرایند کرد.
- باین‌حال، در BDD هنوز آزمون مصنوع اصلی است.
- SDD مشخصه را مرکز قرار می‌دهد؛ کد اجرا و آزمون تأیید می‌کند.

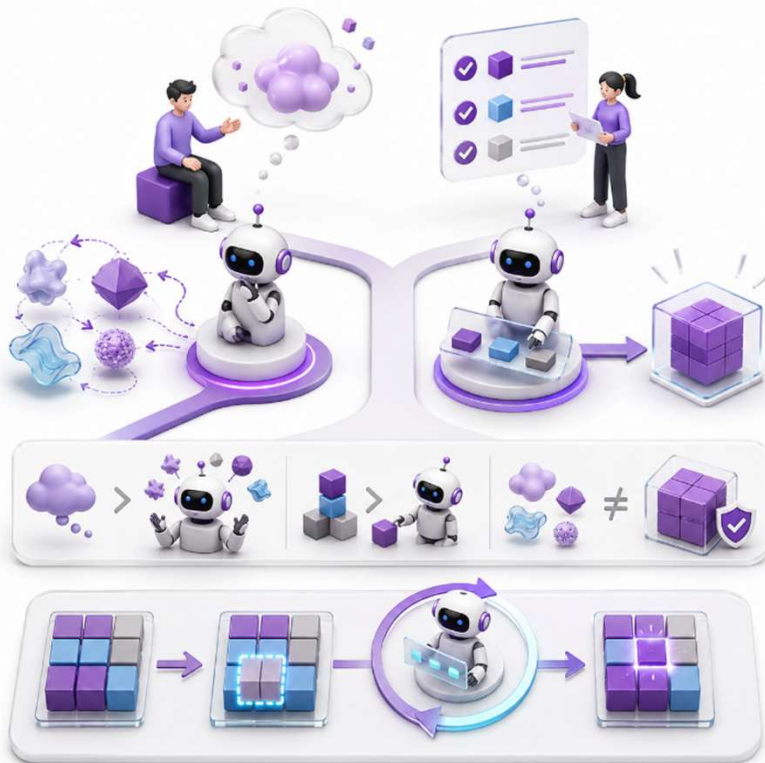


چرا اکنون؟

- عامل‌های هوشمند سریع می‌سازند، اما خلأها را نیز سریع پر می‌کنند.
- توسعه‌دهنده سؤال می‌پرسد؛ عامل اغلب محتمل‌ترین فرض را انتخاب می‌کند.
- پس کیفیت زمینه و دستورها از همیشه مهم‌تر شده است.
- رشد ابزارهای SDD نشان می‌دهد این رویکرد از مرحله آزمایش عبور کرده است.
- مشخصه نسخه‌پذیر، ردیابی تصمیم و ممیزی را نیز ممکن می‌کند.



جدولی که همه چیز را خلاصه می‌کند



- Vibe Coding با پرامپت اجرا شروع می‌شود؛ SDD با مشخصه رفتار.
- در اولی عامل «چه» و «چگونه» را تعیین می‌کند.
- در دومی «چه» از قبل روشن است و عامل «چگونه» را انتخاب می‌کند.
- خروجی Vibe Coding متغیر است؛ خروجی SDD با مشخصه سنجیده می‌شود.
- با تغییر تصمیم، مشخصه اصلاح می‌شود و فقط بخش لازم باز اجرا می‌گردد.

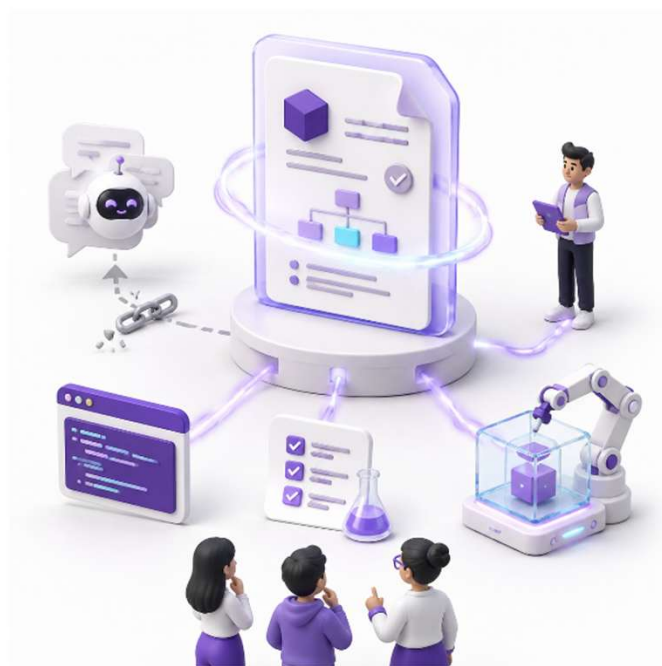


SDD چه مسائلی را حل نمی‌کند؟

- SDD نیازمند دانستن مقصد محصول است.
- ایده مبهم با قالب رسمی، خودبه‌خود شفاف نمی‌شود.
- مشخصه بد می‌تواند محصول اشتباه را کاملاً درست اجرا کند.
- SDD جای فکرکردن را نمی‌گیرد؛ فکر را منظم می‌کند.
- وقتی هنوز مقصد روشن نیست، باید ابتدا نمونه‌سازی و اکتشاف کرد.



تعریفی که ارزش به‌خاطر سپردن دارد

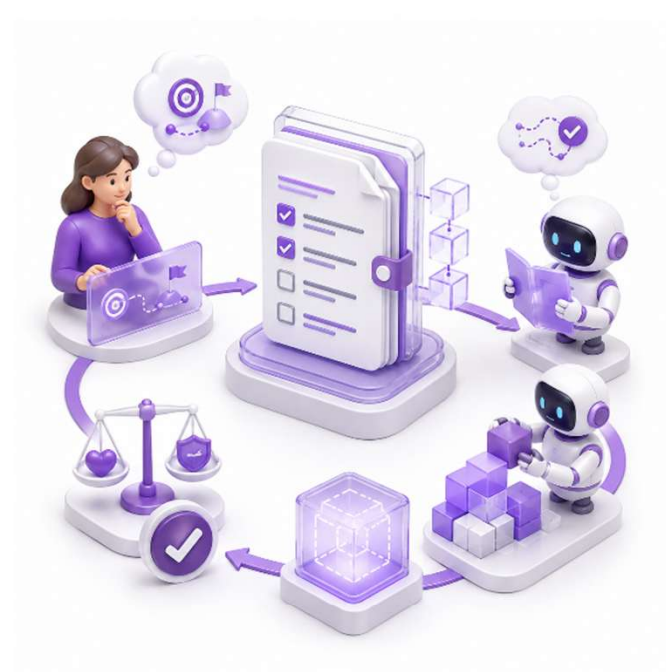


- رفتار مورد انتظار پیش از آغاز پیاده‌سازی تعریف می‌شود.
- این رفتار در مصنوعی نوشته و نسخه‌بندی‌شده باقی می‌ماند.
- عامل آن را می‌خواند، دنبال می‌کند و کارش را با آن می‌سنجد.
- عامل اجرا می‌کند و انسان درباره مقصد و مرزها تصمیم می‌گیرد.
- فصل بعد نشان می‌دهد این مصنوع دقیقاً چه شکلی دارد.



مشخصه به عنوان مصنوع اصلی

- مصنوع مستقل از گفت‌وگویی است که آن را ایجاد کرده است.
- کد، آزمون و خروجی استقرار نمونه‌های آشنای مصنوع هستند.
- مشخصه درست نیز پس از بسته‌شدن گفت‌وگو باقی می‌ماند.
- عضو جدید تیم می‌تواند بدون حافظه قبلی آن را بفهمد.
- مشخصه خلاصه برنامه نیست؛ منبع حقیقت آن چیزی است که می‌سازیم.



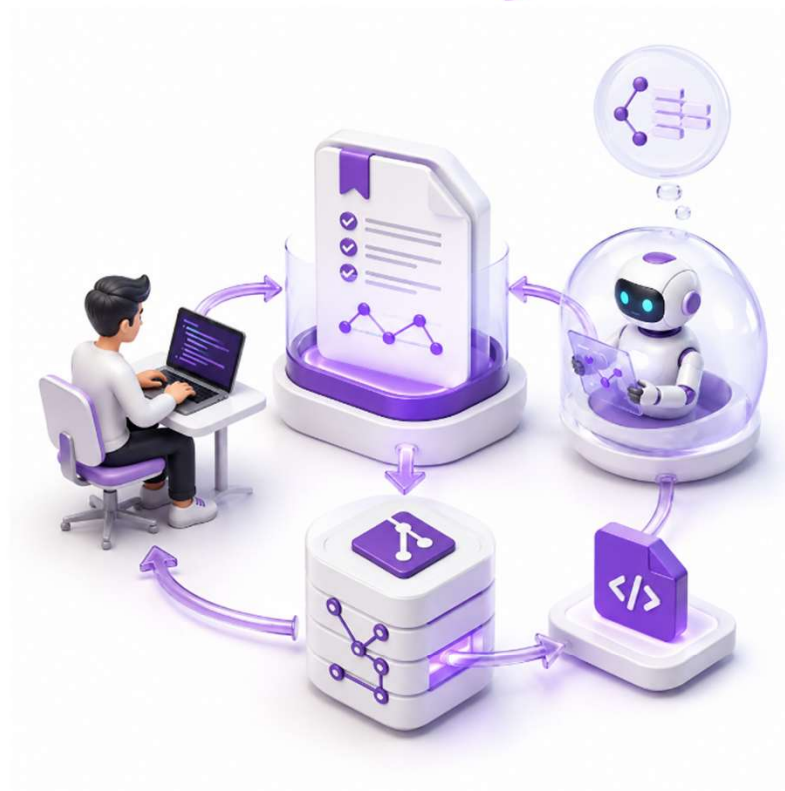
چرا مشخصه‌های سنتی شکست می‌خورند؟

- مشخصه‌های سنتی معمولاً فقط در ابتدای پروژه نوشته می‌شوند.
- نیازها تغییر می‌کنند، اما سند به‌روز نمی‌شود.
- تیم به سند قدیمی اعتماد نمی‌کند و مرجع واقعی به کد منتقل می‌شود.
- در پایان، سند سامانه‌ای را توصیف می‌کند که وجود ندارد.
- مسئله خود مشخصه نیست؛ زمان و مسئول نگهداری آن است.



مشخصه زنده

- مشخصه زنده یک قرارداد فعال است، نه سند نیازمندی بایگانی‌شده.
- در همان مخزن کد نگهداری و با Git نسخه‌بندی می‌شود.
- هر تغییر رفتاری ابتدا در مشخصه و سپس در کد اعمال می‌شود.
- عامل در هر نشست، تداوم زمینه را از این مصنوع می‌گیرد.
- نوشتن مشخصه در عمل طراحی زمینه کاری عامل است.



رفتار، نه معماری

- مشخصه باید بگوید چه چیزی باید درست باشد، نه چگونه ساخته شود.
- «توکن قبلی پس از خروج معتبر نیست» یک رفتار قابل سنجش است.
- «از JWT و Redis استفاده شود» نسخه پیاده سازی را تحمیل می کند.
- تعریف رفتار، فضای تصمیم فنی را برای عامل باز می گذارد.
- تعریف معماری می تواند عامل را مجری دقیق تصمیم اشتباه کند.



مشخصه شامل، چه چیزهایی است و چه چیزهایی نیست؟

- مشخصه کاربر، هدف و اعمال قابل انجام را روشن می‌کند.
- جریان اصلی، خطاها و موارد خارج از دامنه را مشخص می‌کند.
- معیارهای پذیرش باید نتیجه‌ای قابل اثبات یا رد بسازند.
- فرضیات، واقعیت فعلی سامانه را وارد زمینه می‌کنند.
- تخمین زمان، شبه‌کد و ترجیح فناوری معمولاً در مشخصه جایی ندارند.



قرارداد میان شما و عامل



- انسان رفتار مورد انتظار را تعریف می‌کند.
- عامل چیزی می‌سازد که باید آن رفتار را برآورده کند.
- نتیجه به‌جای سلیقه، با معیارهای مشخصه بررسی می‌شود.
- عدم انطباق به یک معیار دقیق اشاره می‌کند، نه حس مبهم نارضایتی.
- این قرارداد، گفت‌وگوی اصلاح را روشن و قابل اقدام می‌کند.



نوشتن مشخصه چقدر زمان می‌برد؟

- یک قابلیت متوسط معمولاً به بیست دقیقه تا یک ساعت مشخصه‌نویسی نیاز دارد.
- بهتر است کار با Grilling آغاز شود، نه با صفحه خالی.
- عامل ابتدا پرسش‌های رفتاری، مرزی و خطاها را مطرح می‌کند.
- پاسخ‌ها مواد اولیه مشخصه را آماده می‌کنند.
- زمان پرسش‌گری، چندین دور اصلاح پرهزینه را حذف می‌کند.



کوتاه‌ترین مشخصه‌ای که می‌توانید بنویسید

- مشخصه فقط باید تصمیم‌های غیرقابل‌واگذاری را روشن کند.
- طول بیشتر الزاماً به معنای وضوح بیشتر نیست.
- سند بلند، نویز زمینه را برای عامل افزایش می‌دهد.
- هدف پیشنهادی برای قابلیت متوسط حدود پانصد واژه است.
- مشخصه نقشه است و Issueها راهنمای گام‌به‌گام هستند.



پیش از ادامه

- تا اینجا، مشخصه به عنوان مصنوع زنده تعریف شد.
- تمرکز آن بر رفتار است، نه معماری و جزئیات اجرا.
- شش مفهوم اصلی، مرز و معیارهای کار را می‌سازند.
- مشخصه باید کوتاه، قابل اعتماد و هم‌نسخه با کد باشد.
- از بخش بعد، این نظریه وارد جریان اجرایی می‌شود.



هفت مرحله توسعه با هوش مصنوعی



- این هفت فاز، لحظه‌های منطقی یک پروژه جدی با عامل هوشمند هستند.
- ترتیب آن‌ها از ابهام به اجرا و سپس تأیید حرکت می‌کند.
- همه فازها الزاماً در هر پروژه اجرا نمی‌شوند.
- حذف هر فاز پیامدی مشخص برای ریسک و کیفیت دارد.
- هدف، ساخت یک جریان قابل کنترل است؛ نه تشریفات بیشتر.



مرحله اول؛ ایده

- شروع می‌تواند یک محصول، قابلیت، باگ یا بازآرایی باشد.
- اندازه مهم نیست؛ تغییر مورد انتظار باید مشخص باشد.
- دام اصلی، شروع اجرا پیش از بیرون‌کشیدن زمینه ذهنی است.
- محدودیت‌ها و تصمیم‌های ضمنی باید آشکار شوند.
- اگر ایده در دو جمله بی‌ابهام بیان نمی‌شود، هنوز آماده نیست.



مرحله دوم؛ تحقیق



- تحقیق زمانی لازم است که فناوری، API یا دامنه‌ای ناشناخته وجود دارد.
- امکان‌پذیری باید پیش از تثبیت مشخصه بررسی شود.
- یافته‌ها در `research.md` به‌عنوان حافظه موقت ثبت می‌شوند.
- این فایل محدودیت‌ها، نمونه‌های معتبر و نکات عملی را نگه می‌دارد.
- تحقیق مقاله نیست؛ کش موقتی برای تصمیم‌گیری دقیق‌تر است.

مرحله سوم؛ نمونه سازی

- نمونه سازی پیش از مشخصه انجام می شود، نه پس از آن.
- هدف، کشف طراحی، تجربه و جهت معماری با قضاوت انسانی است.
- نمونه باید سریع، قابل دورریختن و مناسب اکتشاف باشد.
- لازم نیست کامل کار کند؛ باید تصمیم های مبهم را روشن کند.
- ثبت Commit نمونه، آن را به مرجعی برای مشخصه و عامل تبدیل می کند.



مرحله چهارم؛ سند نیازمندی محصول

- PRD مصنوع مرکزی و مشخصه مقصد است.
- کاربر، رفتار، محدودیت، فرض و معیار پایان را تعریف می‌کند.
- PRD نباید مسیر فنی پیاده‌سازی را تجویز کند.
- دشمن این فاز، وسوسه شروع کدنویسی پیش از تعریف «تمام‌شدن» است.
- چند دقیقه دقت در PRD، ساعت‌ها اصلاح بعدی را کم می‌کند.



مرحله پنجم؛ کانبان

- PRD مقصد را تعریف می‌کند و Kanban مسیر رسیدن را.
- هر Issue باید در یک نشست و با زمینه کامل اجراشدنی باشد.
- تقسیم کار بر اساس Vertical Slice انجام می‌شود، نه لایه‌های فنی.
- Tracer Bullet‌های پرریسک پیش از کارهای آسان قرار می‌گیرند.
- روابط مسدودکننده ترتیب و فرصت اجرای موازی را آشکار می‌کنند.



مرحله ششم؛ چرخه اجرا

- عامل Issue را می‌خواند، اجرا می‌کند، گزارش می‌دهد و می‌بندد.
- در کار کم‌ریسک، چرخه می‌تواند تا حد زیادی خودکار باشد.
- در منطق حساس، بازبینی انسان میان Issueها ضروری است.
- خواندن کد برای حفظ قضاوت فنی انجام می‌شود، نه ریزمدیریت.
- مسئولیت نتیجه همچنان بر عهده تیم انسانی است.



مرحله هفتم؛ تضمین کیفیت

- QA تطابق رفتار واقعی با PRD را بررسی می‌کند.
- عامل می‌تواند از روی PRD و کد، برنامه‌آزمون بسازد.
- انسان سناریوها و حالت‌های مرزی را اجرا می‌کند.
- هر شکست به Issue تازه‌ای در چرخه اجرا تبدیل می‌شود.
- QA آخرین فعالیت نیست؛ معیار خروج از کار است.



چه زمانی می‌توان بعضی مراحل را حذف کرد؟



- تحقیق در صورت نبود مجهول فنی قابل حذف است.
- نمونه‌سازی در صورت روشن‌بودن قضاوت طراحی قابل حذف است.
- PRD، Kanban، چرخه اجرا و QA اجزای الزامی روش هستند.
- حذف PRD و Kanban عملاً بازگشت به Vibe Coding است.
- حذف QA یعنی اعتماد بدون شواهد به تفسیر عامل.



جریان کامل توسعه

- Idea مسئله را روشن می‌کند و Research مجهول‌ها را کاهش می‌دهد.
- Prototype قضاوت طراحی را پیش از تعهد ثبت می‌کند.
- PRD مقصد را تعریف و Kanban آن را به واحدهای اجرا تبدیل می‌کند.
- Execution Loop کار را Issue به Issue پیش می‌برد.
- QA نتیجه را می‌سنجد و شکست‌ها را به چرخه بازمی‌گرداند.



سند نیازمندی محصول؛ مشخصه‌ای که هوش مصنوعی اشتباه برداشت نمی‌کند



- PRD مهم‌ترین سند در یک پروژه AI-assisted است.
- Issue از PRD و کد از Issue ها ساخته می‌شوند.
- آزمون‌ها باید وعده‌های PRD را بررسی کنند.
- خطای PRD به همهٔ مصنوعات پایین‌دست منتقل می‌شود.
- پس کیفیت اجرا پیش از کد، در کیفیت مشخصه تعیین می‌گردد.

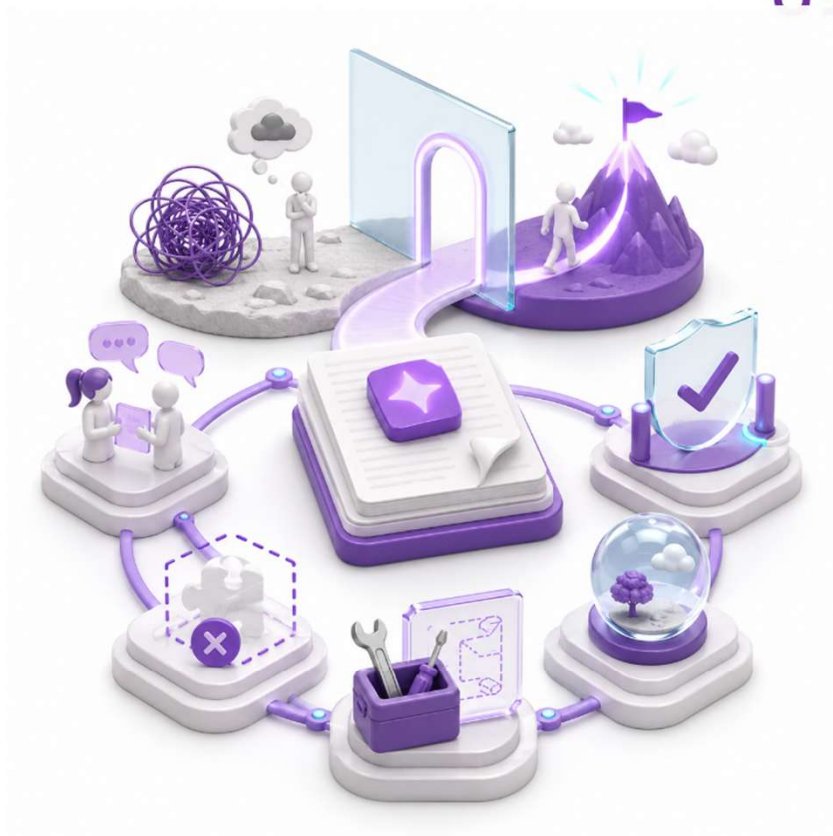


پیش از نوشتن؛ پرسش‌گری عمیق

- نوشتن PRD با پرسش آغاز می‌شود، نه با تولید متن.
- از عامل بخواهید پیش از راه‌حل، دست‌کم ده پرسش مطرح کند.
- پرسش‌ها حالت‌های مرزی، محدودیت‌ها و فرض‌های پنهان را آشکار می‌کنند.
- هر پاسخ، تصمیمی را از دوش حدس عامل برمی‌دارد.
- شدت Grilling باید متناسب با ریسک قابلیت باشد.



هشت بخش سند نیازمندی محصول،

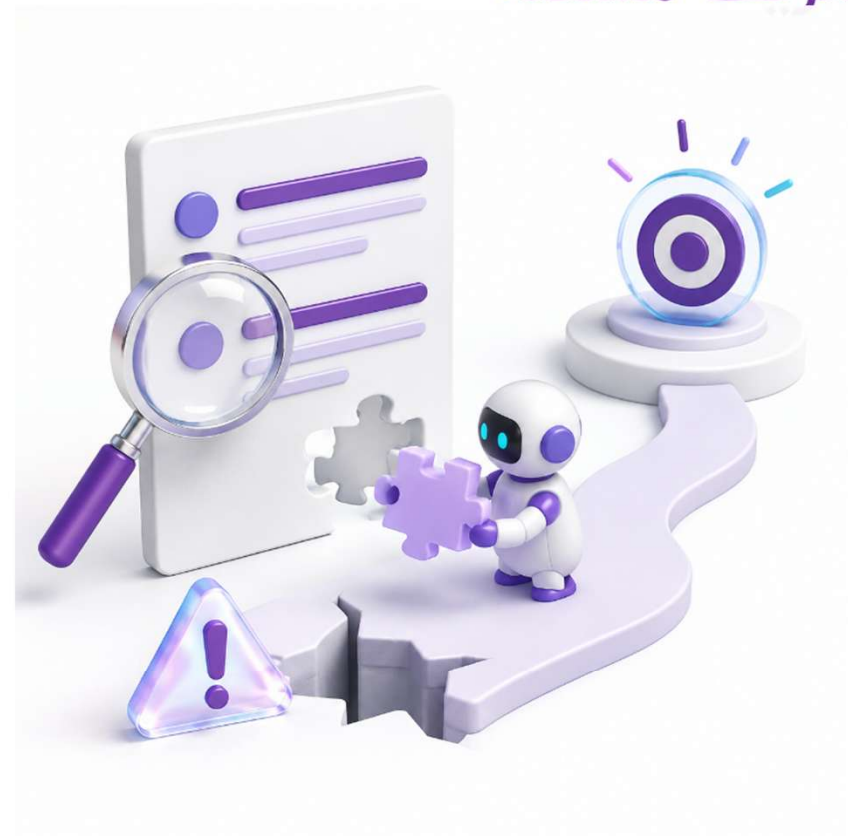


- Problem Statement و Solution مسئله و مقصد را جدا می‌کنند.
- User Stories رفتارهای ملموس کاربر را بیان می‌کنند.
- Constraints و Acceptance Criteria مرز اعتبار نتیجه را می‌سازند.
- Out of Scope و Assumptions دامنه و واقعیت فعلی را روشن می‌کنند.
- Implementation Notes فقط پیشنهاد فنی است و الزام محسوب نمی‌شود.

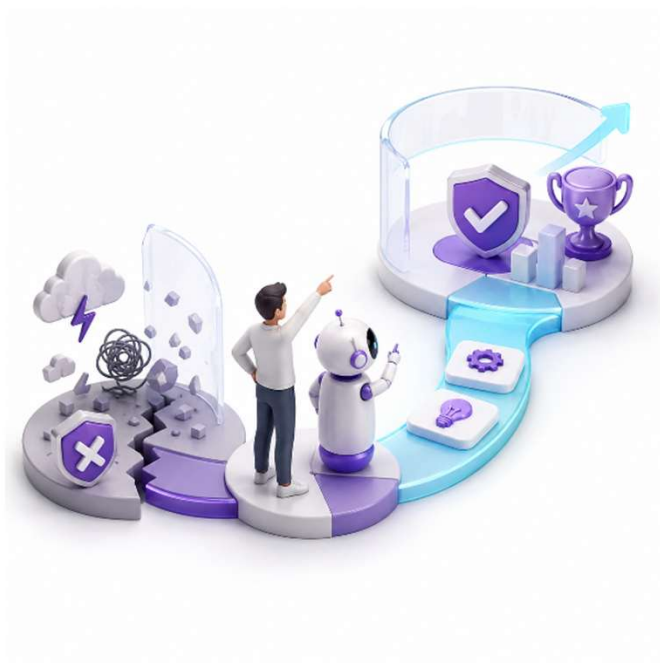


تعریف مسئله

- مسئله باید در یک یا دو جمله بیان شود.
- باید روشن کند چه چیز خراب یا غایب است.
- اهمیت و فوریت مسئله باید قابل فهم باشد.
- Problem Statement راه حل یا آرزوی کلی نیست.
- بیان خوب، مشخص، محدود و دارای پیامد واقعی است.



راه حل



- Solution وضعیت مطلوب پس از پایان کار را توصیف می‌کند.
- زاویه بیان، رفتار کاربر یا سامانه است.
- باید روشن کند چه چیزی در موفقیت و شکست رخ می‌دهد.
- نام فناوری، کتابخانه و الگوی فنی در این بخش ضروری نیست.
- انسان «چه» را تعیین می‌کند و عامل «چگونه» را.



داستان‌های کاربر

- User Story یک عمل مشخص از یک نقش واقعی را بیان می‌کند.
- قالب نقش، اقدام و نتیجه مورد انتظار همچنان مفید است.
- داستان باید رفتاری باشد که بتوان آن را مشاهده کرد.
- برای بیشتر قابلیت‌ها سه یا چهار داستان کافی است.
- بیش از شش داستان معمولاً نشانه ترکیب چند قابلیت است.



محدودیت‌ها

- Constraint محدودیتی است که نقض آن نتیجه را نامعتبر می‌کند.
- محدودیت با ترجیح یا پیشنهاد فنی تفاوت دارد.
- قید امنیتی یا قانونی نمونه مناسبی از Constraint است.
- عبارت باید صریح و بدون امکان تفسیر دوگانه باشد.
- اگر محدودیت واقعی وجود ندارد، این بخش را مصنوعی پر نکنید.



معیارهای پذیرش

- هر معیار پذیرش باید بتواند قبول یا رد شود.
- ورودی، رخداد و نتیجه قابل مشاهده را مشخص کنید.
- وضعیت‌های موفق، ناموفق و منقضی را جداگانه پوشش دهید.
- عبارت «سامانه درست کار می‌کند» قابل راستی‌آزمایی نیست.
- معیار دقیق، مبنای آزمون و پایان Issue است.



موارد خارج از دامنه



- Out of Scope کارهایی را که عمدهً انجام نمی‌شوند ثبت می‌کند.
- نبود آن عامل را به گسترش منطقی اما ناخواستهً دامنه تشویق می‌کند.
- قابلیت‌های آینده و Issue‌های جدا باید صریح نام برده شوند.
- این بخش از پیاده‌سازی اضافی و بازکاری جلوگیری می‌کند.
- مرز روشن به اندازه هدف روشن اهمیت دارد.



فرضیات

- Assumptions واقعیت فعلی پروژه را وارد مشخصه می‌کنند.
- عامل تصمیم‌های فنی خود را بر پایه همین واقعیت می‌سازد.
- فرض اشتباه، کد درست برای سامانه‌ای متفاوت تولید می‌کند.
- پشته، ساختار موجود و وضعیت آزمون‌ها نمونه‌های مهم هستند.
- این بخش معمولاً بیشترین اثر را بر کیفیت خروجی دارد.



نکات پیاده‌سازی



- Implementation Notes بخشی اختیاری برای پیشنهادهای فنی است.
- این پیشنهادهای برخلاف Constraints الزام‌آور نیستند.
- عامل می‌تواند با استدلال بهتر، مسیر دیگری انتخاب کند.
- واژه‌هایی مانند «بررسی کن» تفاوت پیشنهاد و دستور را روشن می‌کنند.
- مرزبندی صریح، اختیار فنی را بدون ابهام واگذار می‌کند.



یک سند نیازمندی محصول کامل

- نمونه کامل کتاب مهاجرت احراز هویت به httpOnly cookie است.
- مسئله، رفتار مطلوب و داستان‌های کاربر به هم متصل‌اند.
- Constraints امنیتی از Acceptance Criteria قابل آزمون پشتیبانی می‌کنند.
- Assumptions و Out of Scope از گسترش و حدس جلوگیری می‌کنند.
- کل سند زیر چهارصد واژه، اما برای اجرا کافی است.



سند نیازمندی محصول کجا نگهداری می‌شود؟

- محل عملی PRD همان جایی است که کار اجرایی مدیریت می‌شود.
- در GitHub می‌توان هر PRD را یک Issue والد در نظر گرفت.
- Issueهای فرزند با شماره به PRD ارجاع می‌دهند.
- تاریخچه تغییر، زمان و علت اصلاح را نگه می‌دارد.
- در ابزارهای دیگر نیز نزدیکی PRD و کارهای فرزند اصل کلیدی است.

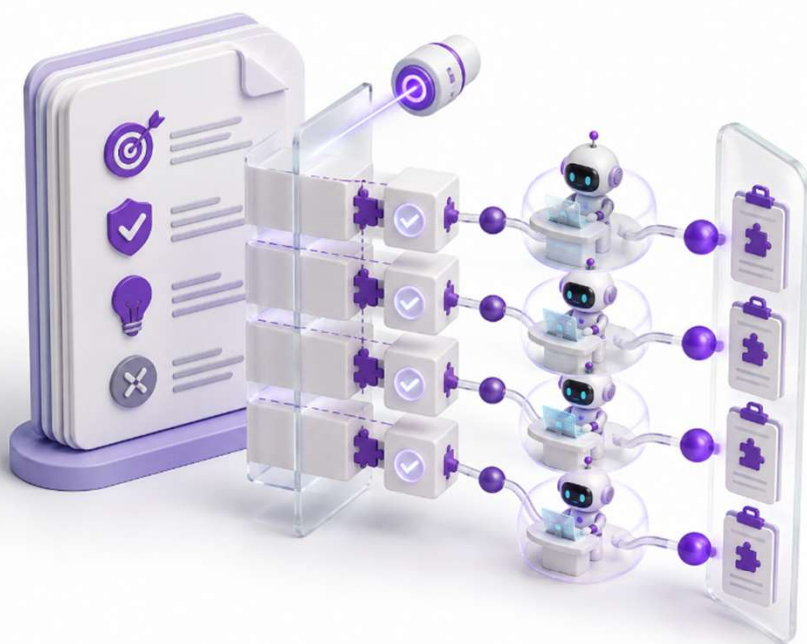


طول مناسب سند

- هدف مناسب برای قابلیت متوسط حدود پانصد واژه است.
- طول بیشتر می‌تواند نشانه ترکیب چند قابلیت باشد.
- تجویز پیاده‌سازی نیز معمولاً PRD را بی‌دلیل طولانی می‌کند.
- جزئیات محلی باید به Issue‌های فرزند منتقل شوند.
- PRD کوتاه‌تر، خواندنی‌تر، به‌روزرسانی‌پذیرتر و سخت‌تر برای بدفهمی است.



از سند نیازمندی محصول تا ایشوها؛ برش‌های عمودی و گلوله‌های ردیاب



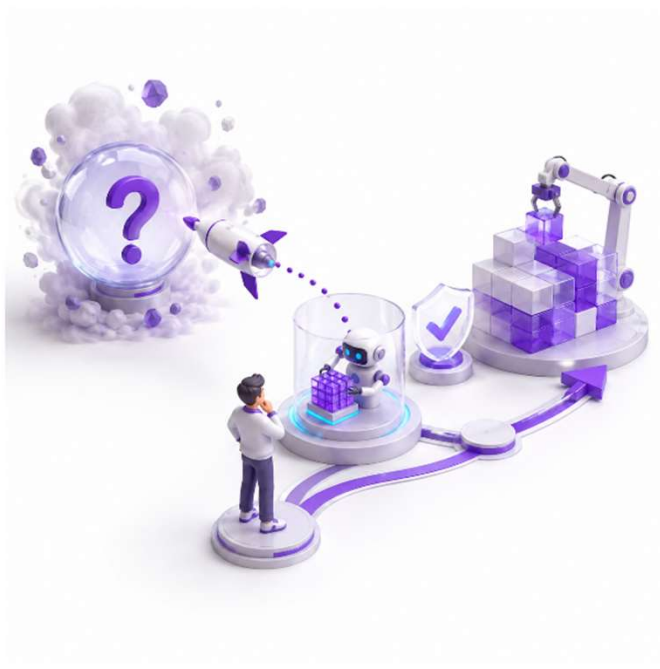
- PRD مقصد، معیارها، فرض‌ها و خارج از دامنه را آماده کرده است.
- دادن کل PRD به عامل هنوز یک چرخه اجرا نمی‌سازد.
- عامل به واحدهای کوچک، مستقل و دارای معیار خروج نیاز دارد.
- هر واحد باید در یک نشست و بدون حافظه نشست قبلی اجرا شود.
- این واحدهای قابل اجرا همان Issue هستند.

برش عمودی چیست؟

- تقسیم به Database، Backend، Frontend و ظاهرآً مرتب اما غیرقابل سنجش است.
- بسته شدن یک لایه لزوماً رفتار قابل استفاده‌ای ایجاد نمی‌کند.
- Vertical Slice یک رفتار را از رابط تا داده کامل می‌کند.
- هر Slice پس از پایان، مستقل قابل مشاهده و آزمون است.
- واحد کار باید قابلیت تحویل دهد، نه صرفاً بخشی از معماری.



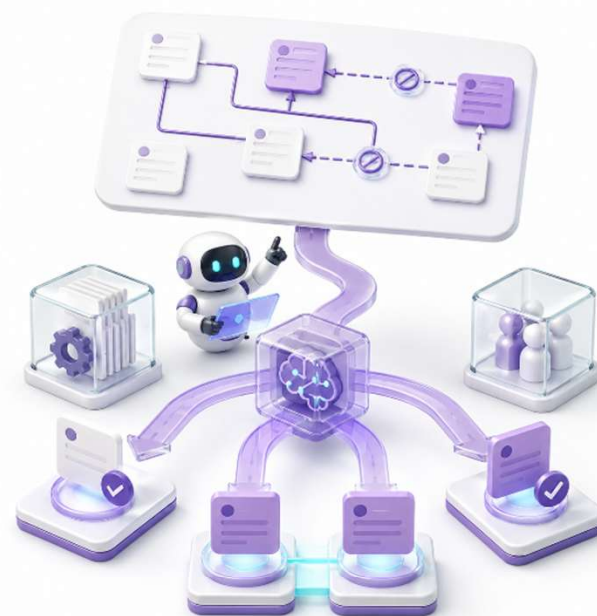
گلوله‌های ردیاب؛ ابتدا سراغ ناشناخته‌ها بروید



- نخستین Issue باید پرریسک‌ترین مجهول را هدف بگیرد.
- Spike فقط برای پاسخ به یک پرسش فنی ساخته می‌شود.
- هدف آن اثبات امکان‌پذیری پیش از سرمایه‌گذاری اصلی است.
- اگر فرض بنیادین غلط باشد، باید در Issue اول آشکار شود.
- ابتدا جهت را می‌سنجیم؛ سپس حجم اصلی کار را اجرا می‌کنیم.

روابط مسدودکننده

- Issue مستقل از حافظه است، اما الزاماً مستقل از ترتیب نیست.
- وابستگی‌های دنباله‌ای باید صریحاً با blocked by ثبت شوند.
- عامل از این روابط ترتیب معتبر اجرا را می‌فهمد.
- Issue‌هایی با والد مشترک و بدون وابستگی متقابل قابل موازی‌سازی‌اند.
- موازی‌سازی نتیجه طراحی خوب برد است، نه یک تصادف اجرایی.



یک ایشوی قابل اجرا به چه چیزهایی نیاز دارد؟



- Issue باید به PRD والد ارجاع دهد.
- شرح آن باید رفتار مشخص این واحد را بیان کند.
- معیارهای پذیرش محلی، پایان کار را قابل تشخیص می‌کنند.
- فقط فرض‌های مرتبط و وابستگی‌های لازم باید آورده شوند.
- دستور پایان، گزارش تصمیم‌ها و بستن Issue را الزام می‌کند.



مقایسه یک ایشوی بد و یک ایشوی خوب

- Issue بد فقط یک دستور کلی مانند «احراز هویت را پیاده کن» می‌دهد.
- زمینه، معیار پایان و مرز کار در آن نامعلوم است.
- Issue خوب رفتار، PRD، معیارها و فرض‌ها را کنار هم می‌آورد.
- وابستگی و اقدام پس از پایان را نیز صریح می‌کند.
- نتیجه، اجرای کم‌سؤال و بدون فرض‌های غیرمجاز است.



چگونه ایشوها را ایجاد کنیم؟

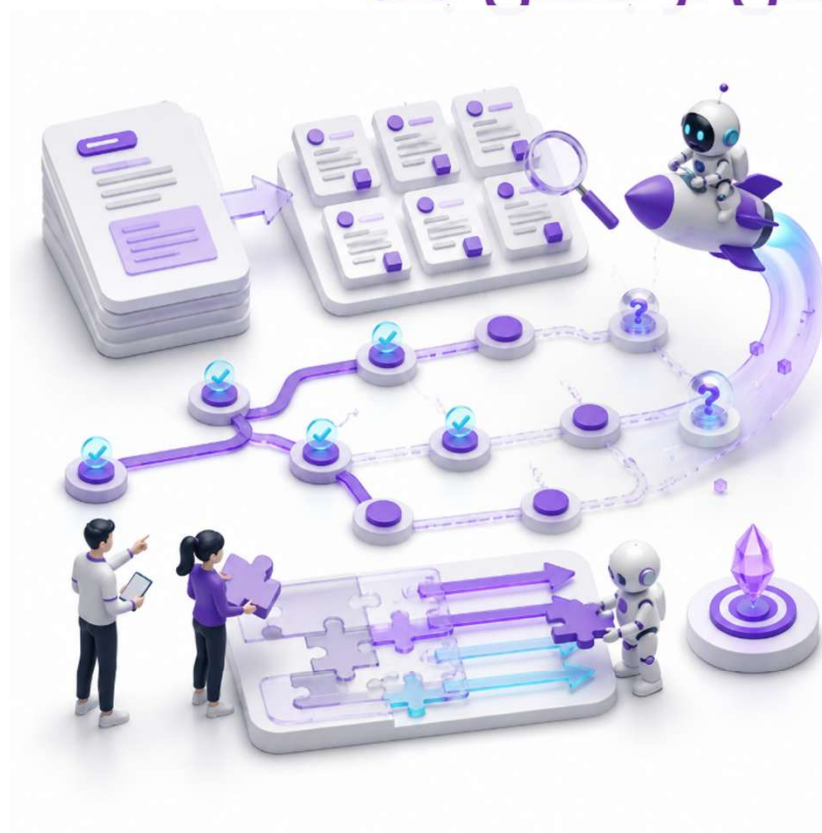


- Issue را می‌توان دستی یا با کمک عامل تولید کرد.
- به عامل PRD، اصل Vertical Slice و روابط مسدودکننده را بدهید.
- از آن بخواهید Tracer Bullet ها را در ابتدای فهرست پیشنهاد کند.
- پیشنهاد عامل باید پیش از ایجاد نهایی بازبینی و اصلاح شود.
- تعیین اولویت ریسک، قضاوتی انسانی و غیرقابل واگذاری است.



پیش، از بخش، بعد

- اکنون PRD به واحدهای کوچک و اجرایی تبدیل شده است.
- هر Issue زمینه، معیار و وابستگی خود را دارد.
- Tracer Bullet ها مجهول‌های پرریسک را زود آزمایش می‌کنند.
- برد ترتیب معتبر و مسیرهای موازی را آشکار کرده است.
- بخش بعد درباره اجرای منضبط همین برد است.



چرخه اجرا



- برد، PRD، وابستگی‌ها و Tracer Bullet‌ها آماده‌اند.
- اکنون عامل وارد اجرا می‌شود، اما تیم ناپدید نمی‌شود.
- واگذاری کامل می‌تواند تصمیم‌های معقول اما نامتناسب تولید کند.
- تعداد Issue‌های بسته‌شده به‌تنهایی نشانه پیشرفت درست نیست.
- قاعده چرخه این است: عامل اجرا می‌کند و انسان نظارت.



قرارداد با عامل

- عامل ابتدا Issue و PRD والد را می‌خواند.
- پس از اجرا، خلاصه تغییر و تصمیم‌ها را ثبت می‌کند.
- سپس Issue را می‌بندد و به مورد بعدی می‌رود.
- این گزارش، زمینه لازم برای QA و اصلاح آینده را حفظ می‌کند.
- دستور گزارش باید صریحاً در هر Issue نوشته شود.



چرخه رالف

- در Ralph Loop عامل Issue را پیوسته و سری اجرا می‌کند.
- بین Issue مداخله انسانی الزامی نیست.
- این مدل برای دامنه آشنا، معیار روشن و ریسک پایین مناسب است.
- در منطق حساس، خطای کوچک می‌تواند در Issue‌های بعدی انباشته شود.
- خودکارسازی ابزار است؛ سیاست ثابت برای همه کارها نیست.



چرخه چندعاملی

- در مدل چندعاملی، Orchestrator مسیرهای مستقل را شناسایی می‌کند.
- عامل‌های هم‌تا Issue را از یک فهرست مشترک برمی‌دارند.
- هماهنگی از طریق وضعیت مشترک است، نه گفت‌وگوی مستقیم عامل‌ها.
- موازی‌سازی فقط در نبود وابستگی و تداخل ارزش ایجاد می‌کند.
- با افزایش موازی‌سازی، ثبت تصمیم هر Issue حیاتی‌تر می‌شود.



چه زمانی باید مداخله کنیم؟

- پرسش عامل نشانه شکاف در Issue است و باید مستند شود.
- تصمیم غیرمنتظره در گزارش، نقطه مناسب بازبینی است.
- پیش از Issue وابسته، فرض‌های ایجادشده در Issue قبلی را بررسی کنید.
- عبور از چند Issue بدون خواندن کد، ریسک انباشته می‌سازد.
- مداخله برای حفظ قضاوت است، نه کنترل هر حرکت عامل.



انسان همیشه باید کد را بخواند

- پیش از پایان هر نشست، فردی با قضاوت فنی کد را می‌خواند.
- کد می‌تواند آزمون‌ها را پاس کند و هنوز الگوی نامناسبی بسازد.
- معیار پذیرش رفتار را می‌سنجد؛ Code Review کیفیت تصمیم را.
- خواندن کد مانع از دست‌رفتن فهم فنی پروژه می‌شود.
- این کار بی‌اعتمادی نیست؛ مسئولیتی است که عامل نمی‌تواند بپذیرد.



مدیریت زمینه میان نشست‌ها با GSD



- پروژه طولانی زمینه را میان نشست‌ها پراکنده می‌کند.
- GSD در پایان نشست، وضعیت جاری را ثبت می‌کند.
- Decisions و Completed، In Progress، Pending چهار بخش اصلی‌اند.
- نشست بعد با خواندن این وضعیت، بدون بازسازی حدسی آغاز می‌شود.
- چند دقیقه Context Hygiene، زمان و خطای شروع بعدی را کم می‌کند.



تضمین کیفیت به عنوان یک ایشو، نه اصلاح لحظه آخر

- آخرین Issue برد باید همیشه QA باشد.
- QA فقط اجرای تست خودکار نیست؛ تجربه واقعی را با PRD می‌سنجد.
- عامل برنامه آزمون و حالت‌های مرزی را پیشنهاد می‌کند.
- انسان برنامه را اجرا و شکست‌ها را به Issue تبدیل می‌کند.
- کار با عبور از QA تمام می‌شود، نه با خالی شدن برد.



نمای کامل چرخه اجرا



- عامل PRD و Issue را می‌خواند، اجرا می‌کند و گزارش می‌دهد.
- انسان کد و تصمیم‌ها را پیش از ادامه بررسی می‌کند.
- Ralph Loop این توالی را سری و Agent Teams آن را موازی می‌کند.
- پس از بسته شدن همه Issue ها، QA آغاز می‌شود.
- شکست QA Issue تازه می‌سازد و چرخه دوباره ادامه می‌یابد.



اجرای ساخت‌یافته SDD با GitHub SpecKit

- SpecKit روشی ساخت‌یافته برای اجرای SDD در مقیاس است.
- از عامل‌ها و محیط‌های مختلف توسعه پشتیبانی می‌کند.
- مصنوعات را در ساختار ثابت و قابل‌پیش‌بینی نگه می‌دارد.
- فرمان‌های استاندارد، جریان ساخت مشخصه تا اعتبارسنجی را هدایت می‌کنند.
- این بخش روش، ساختار و زمان مناسب استفاده را توضیح می‌دهد.



SpecKit چیست؟



- SpecKit جایگزین Issue، PRD، و چرخه اجرا نیست.
- این ابزار همان روش را رسمی و تکرارپذیر می‌کند.
- ساختار فایل استاندارد، محل هر مصنوع را مشخص می‌سازد.
- فرمان‌های مشخص، پرامپت‌های طولانی و متغیر را کاهش می‌دهند.
- Constitution اصول غیرقابل مذاکره پروژه را به همه مراحل تزریق می‌کند.



نصب و ساختار پوشه‌ها

- SpecKit با ابزار uv نصب و در مخزن مقاردهی اولیه می‌شود.
- گزینه integration عامل مورد استفاده را تعیین می‌کند.
- `/constitution.md/memoryspecify.` اصول پروژه را نگه می‌دارد.
- هر قابلیت در specs دارای `plan.md`، `spec.md` و `tasks.md` است.
- پس از پایان قابلیت، این فایل‌ها به‌عنوان مستند دائمی باقی می‌مانند.



قانون اساسی پروژه

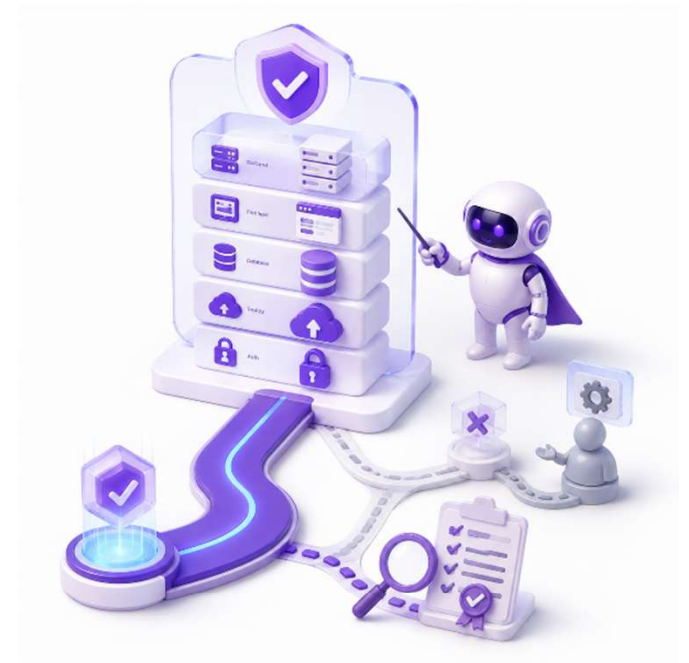


- Constitution رفتار عامل را نمی‌گوید؛ اصول فنی پروژه را اعلام می‌کند.
- فناوری مجاز، امنیت، کارایی، کدنویسی و حاکمیت در آن ثبت می‌شوند.
- Plan و Spec باید پیش از تولید با این اصول سازگار باشند.
- انحراف ممکن است، اما نیازمند دلیل صریح و بازبینی است.
- این فایل زمینه پایدار مشترک در همه نشست‌ها ایجاد می‌کند.



استانداردهای فناوری

- این بخش پلتفرم‌ها، Framework ها و Stack های تأییدشده را ثبت می‌کند.
- Backend، Frontend، Database، Deploy و Auth می‌توانند صریح شوند.
- عامل این انتخاب‌ها را مسیر پیش‌فرض می‌داند.
- جایگزین فنی بدون دلیل پیشنهاد نمی‌شود.
- هر انحراف باید آشکار، مستدل و قابل بازبینی باشد.



الزامات امنیتی

- الزامات امنیتی بدون استثنا در این بخش قرار می‌گیرند.
- احراز همه درخواست‌ها می‌تواند یک اصل دائمی باشد.
- PII نباید در Log ثبت و Secret نباید در کد ذخیره شود.
- محل نگهداری Session Token نیز باید صریح باشد.
- عامل راه ساده‌تر اما ممنوع را پیشنهاد نخواهد کرد.



کارایی و مقیاس پذیری

- بودجه کارایی و ظرفیت در این بخش قابل سنجش می شود.
- زمان بارگذاری صفحه باید با شرایط شبکه تعریف شود.
- حد پاسخ Query باید عدد و واحد مشخص داشته باشد.
- ظرفیت همزمانی باید متناسب با سناریوی عرضه تعیین گردد.
- این قیود هنگام طراحی هر قابلیت در زمینه عامل حضور دارند.



استانداردهای کدنویسی



- این بخش قراردادهای پایدار تولید کد را تعیین می‌کند.
- حداقل پوشش آزمون برای منطق حساس قابل تعریف است.
- قاعده نام‌گذاری و زبان Commentها یکدست می‌شود.
- محدودیت طول Function یا استفاده از any می‌تواند ثبت شود.
- بدون این اصول، سبک خروجی عامل میان نشست‌ها تغییر می‌کند.



انطباق، و حاکمیت

- الزامات قانونی، دسترس‌پذیری و ممیزی در این بخش ثبت می‌شوند.
- سطح WCAG باید دقیق و قابل ارزیابی باشد.
- فعالیت‌های حساس می‌توانند نیازمند Audit Log باشند.
- رضایت و جمع‌آوری داده باید با قواعد حریم خصوصی سازگار شود.
- اهمیت این بخش با حساسیت داده و تعداد کاربران افزایش می‌یابد.



پنج فرمان جریان اصلی



- `/speckit.specify` مشخصه قابلیت را تولید می‌کند.
- `/speckit.plan` مشخصه را به طرح فنی تبدیل می‌کند.
- `/speckit.task` طرح را به کارهای اجرایی می‌شکند.
- `/speckit.implement` کارها را با زمینه کامل اجرا می‌کند.
- `/speckit.validate` نتیجه را با معیارهای مشخصه می‌سنجد.



فرمان تولید مشخصه: /speckit.specify

- این فرمان توصیف رفتار مطلوب را به spec.md تبدیل می‌کند.
- مسئله، راه‌حل، داستان‌ها و محدودیت‌ها ساختار می‌یابند.
- معیارهای پذیرش، خارج از دامنه و فرضیات نیز ثبت می‌شوند.
- خروجی پیش از نهایی‌شدن با Constitution تطبیق داده می‌شود.
- Grilling قبلی، شکاف‌های مشخصه تولیدشده را کاهش می‌دهد.



فرمان طراحی برنامه فنی: /speckit.plan



- این فرمان از روی Spec یک plan.md فنی تولید می‌کند.
- معماری، Component‌های تغییرپذیر و وابستگی‌ها مشخص می‌شوند.
- تصمیم‌های فنی مرتبط در Plan ثبت می‌گردند.
- Plan نمی‌تواند اصول Constitution را نادیده بگیرد.
- بدین ترتیب «چگونه» پس از تثبیت «چه» طراحی می‌شود.



فرمان تولید وظایف: /speckit.task

- این فرمان Spec و Plan را به tasks.md تبدیل می‌کند.
- Taskها بر اساس Vertical Slice تقسیم می‌شوند.
- روابط مسدودکننده به صورت صریح ثبت می‌گردند.
- Tracer Bulletهای پریسک شناسایی و مقدم می‌شوند.
- خروجی خودکار است، اما ترتیب و دامنه باید بازبینی شود.



فرمان پیاده‌سازی: `/speckit.implement`



- این فرمان Task منتخب را اجرا می‌کند.
- عامل همزمان Spec، Plan و Constitution را می‌خواند.
- کد با زمینه کامل و محدودیت‌های تثبیت‌شده تولید می‌شود.
- تصمیم‌ها و تغییرهای انجام‌شده مستند می‌گردند.
- تفاوت اصلی با پرامپت عمومی، حذف فرض‌های بدون پشتوانه است.



فرمان اعتبارسنجی: /speckit.validate

- این فرمان کد نهایی را با Acceptance Criteria مقایسه می‌کند.
- گزارش، موارد Passed، Failed و نیازمند Review را جدا می‌سازد.
- شکست‌ها باید به معیار و محل مرتبط اشاره کنند.
- این مرحله مشکلات آشکار را پیش از بازبینی انسانی پیدا می‌کند.
- Validate کمک QA است و جای قضاوت انسان را نمی‌گیرد.



سه فرمان کنترل کیفیت



- سه فرمان کیفیت، شکاف و ناسازگاری را پیش از اجرا کاهش می‌دهند.
- `/speckit.clarify` ابهام‌های Spec را به سؤال تبدیل می‌کند.
- `/speckit.analyze` سازگاری Spec، Plan، Task و Constitution را می‌سنجد.
- `/speckit.checklist` فهرست کنترل مخصوص همان قابلیت می‌سازد.
- ارزش اصلی آن‌ها انتقال کشف خطا به مرحله ارزان تحلیل است.



فرمان رفع ابهام: /speckit.clarify

- Clarify تحلیل شکاف مشخصه را به صورت پرسش انجام می دهد.
- ابهام رفتار، خطا و تعامل میان قابلیت ها آشکار می شود.
- فرمان را می توان پس از هر دور پاسخ دوباره اجرا کرد.
- چرخه تا زمانی ادامه می یابد که پرسش مهمی باقی نماند.
- این فرمان شکل نظام مند Grilling درون SpecKit است.



فرمان تحلیل سازگاری: /speckit.analyze

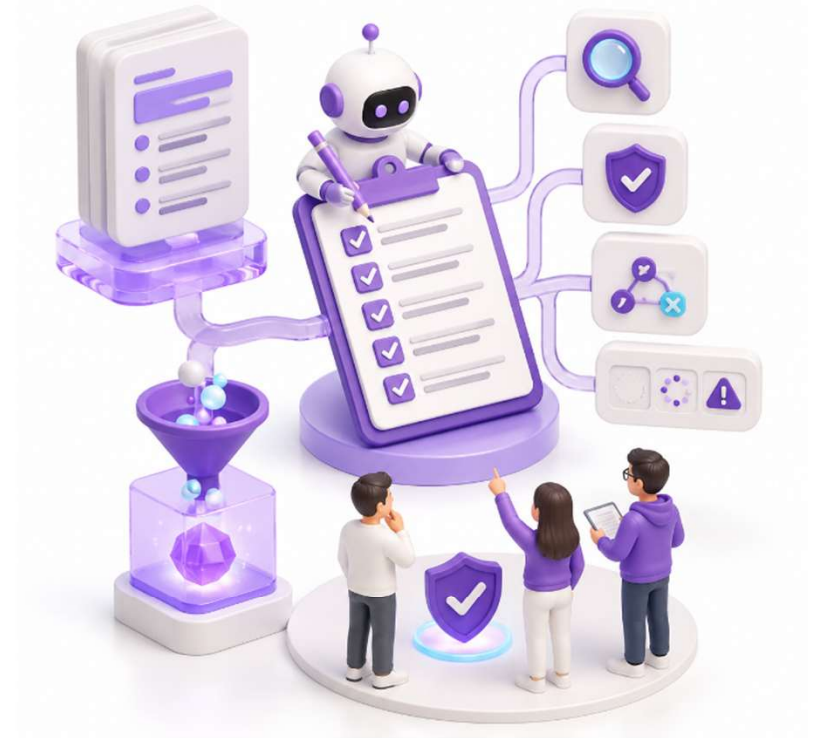


- Analyze پوشش همه نیازهای Spec در Plan را بررسی می‌کند.
- همچنین پوشش Component های Plan در Task ها را می‌سنجد.
- تعارض Plan با Constitution را پیش از اجرا گزارش می‌دهد.
- معیار پذیرش بدون مسیر پیاده‌سازی نیز شناسایی می‌شود.
- کشف ناسازگاری در تحلیل بسیار ارزان‌تر از کشف آن در کد است.



فرمان تولید چک لیست: /speckit.checklist

- Checklist عمومی نیست و از محتوای همان Spec ساخته می‌شود.
- پوشش آزمون برای هر معیار پذیرش را پرس‌وجو می‌کند.
- وجود رفتار خطا در همه جریان‌ها را بررسی می‌کند.
- حالت‌های Empty، Loading و Error را به یاد تیم می‌آورد.
- این پرسش‌ها کیفیت Spec را پیش از اجرا کامل‌تر می‌کنند.



راهبرد شاخه‌ها؛ مشخصه همراه کد حرکت می‌کند



- هر قابلیت در Branch اختصاصی خود توسعه می‌یابد.
- Spec، Plan، Tasks و کد همان قابلیت کنار هم قرار می‌گیرند.
- Pull Request همه این مصنوعات را یکجا برای Review ارائه می‌کند.
- Reviewer هم وعده، هم طرح و هم اجرای واقعی را می‌بیند.
- این هم‌مکانی نیاز به همگام‌سازی شفاهی را کاهش می‌دهد.



چه زمانی، از SpecKit استفاده کنیم؟

- برای پروژه‌های چند هفته‌ای و چند نفره، ساختار SpecKit ارزشمند است.
- قابلیت پیچیده معمولاً به Plan مستقل از Spec نیاز دارد.
- اگر مصنوعات باید در مخزن و استاندارد باشند، SpecKit مناسب است.
- برای Prototype و محصول دورریختنی، این ساختار سرشار ایجاد می‌کند.
- پروژه بسیار کوچک یا نیازمند آزادی کامل، ابزار سبک‌تری می‌خواهد.

